

FPGA Acceleration of Convolution-based Classification Algorithms for Streaming Time Series

Anonymous Authors

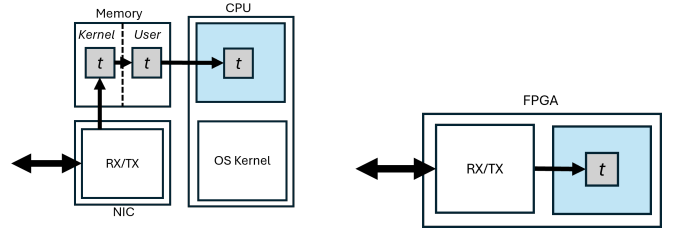
Abstract—This paper presents FPGA accelerator architectures for two time series classification algorithms, MiniRocket and Hydra, both of which employ convolutional kernels to generate features for downstream classifiers. Prior work has established that a combined MultiRocket+Hydra classifier is competitive with the state of the art in this domain; our accelerators match reference CPU accuracy within 0.01% on all benchmark datasets. In single-node evaluation, MiniRocket achieves 8.5–58.7 $\times$ speedup over an optimized C++ baseline with three compute units; at batch-size-one inference, the 3-CU FPGA exceeds a Tesla T4 GPU on all evaluated datasets for both algorithms while consuming 25.7 W vs. 54–68 W. We additionally evaluate an on-chip dataflow decomposition for Hydra via AXI-Stream interfaces, demonstrating that its 512-kernel architecture can be partitioned into independently synthesized sub-kernels while preserving bit-exact accuracy. In a separate network streaming experiment, the FPGA configured as a SmartNIC achieves 1.37 $\times$ lower round-trip time than a DPDK-optimized CPU baseline. These results demonstrate how and when FPGAs retain an advantage over CPUs and GPUs for processing streaming time series data in real time.

I. INTRODUCTION

A **time series** is an ordinal sequence of data points which have been sampled at a consistent rate over some period of time. **Time series classification** fits a **model** which maps the time series, or a subsequence of the time series, to a discrete **response variable**. For example, one or more sensors monitoring human physiological data could generate a time series; from the time series it might be possible to classify human behavior such as sleeping, sitting, or walking. Novel machine learning algorithms for time series classification are developed every year [1]–[3], and are evaluated on datasets maintained by academic researchers [4]–[6].

To date, limited attention has been paid to the hardware acceleration of time series classification algorithms. Researchers who develop the algorithms, typically in Python, do not engage in substantive performance tuning beyond, e.g., optimizing hyperparameters. The best performing algorithms in terms of classification accuracy take days to train¹ on relatively limited datasets, e.g., compared to ImageNet in computer vision [8], and, in response, have developed more efficient algorithms which sacrifice accuracy for computational efficiency [9]. To the best of our knowledge, there has been no research, to date, on hardware accelerators, either FPGA or ASIC, to accelerate state-of-the-art algorithms in this domain.

FPGAs are a promising solution for real-time streaming classification: an FPGA connected to a network can process datapoints directly from network packets, bypassing the host



(a) CPU network interface via (b) FPGA configured as a SmartNIC and host memory.

Fig. 1: A time series datapoint received by a traditional networked CPU (a) vs. an FPGA SmartNIC (b). The CPU path requires NIC $\rightarrow$ PCIe $\rightarrow$ kernel $\rightarrow$ user-space copies; the FPGA transmits directly from network to accelerator.

memory copies required by CPU/GPU-based systems. Performing packet processing and data analysis together on-chip eliminates these inefficiencies, as shown in Fig. 1.

Two promising time series classification algorithms for acceleration are MiniRocket [9] and Hydra [7]. Both transform an input time series into a set of features that are classified using ridge regression or logistic regression, using convolutional kernels followed by a pooling layer which extracts the features. MiniRocket uses fixed ternary kernels ($\{-1, 2\}$) amenable to zero-DSP multiplication on FPGAs, while Hydra introduces learned random kernels with group-based histogram pooling, representing a complementary design point.

This paper presents FPGA accelerator architectures for MiniRocket and Hydra, evaluated in single-node and network streaming contexts. Key design contributions include dot-product reuse in the Convolution Engine, a fused convolution-pooling kernel eliminating intermediate storage, and a floating-point multiplier specialized to $\{-1, 2\}$ consuming zero DSP slices. MiniRocket achieves 8.5–58.7 $\times$ speedup over optimized C++ with up to three compute units. At batch-size one, the 3-CU FPGA exceeds a Tesla T4 GPU on all datasets for both algorithms while consuming 2.1–2.6 $\times$ less power. Both accelerators match reference CPU accuracy within 0.01%.

II. BACKGROUND: TIME SERIES CLASSIFICATION USING UNTRAINED CONVOLUTIONAL KERNELS

We describe a family of time series classifiers which take inspiration from **random convolutional kernels** in machine learning [10]–[14]: a large number of randomly-generated kernels with fixed weights can capture many relevant features for a downstream classifier. This approach is quite different

¹See, e.g., Ref. [7], Section 2.3.

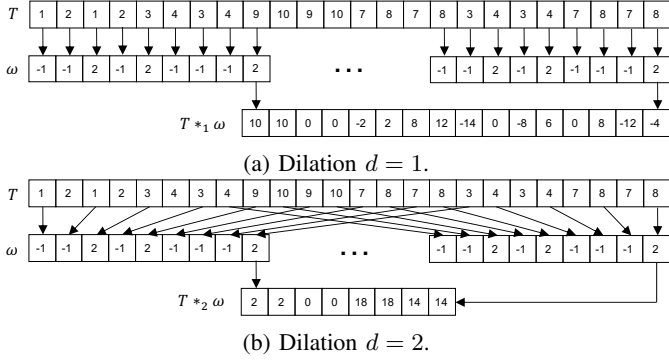


Fig. 2: Convolutions with dilations (a) $d = 1$ and (b) $d = 2$.

from **convolutional neural networks (CNNs)** where kernel weights are learned [15]–[21].

A. Classifier Overview

Let $T = \langle t_1, t_2, \dots, t_n \rangle \in \mathbb{R}^n$ be a time series of length n . T can be **padded** with leading/trailing zeroes as needed. T 's **first-order difference**, $T' = \langle t_i - t_{i-1} | 2 \leq i \leq n \rangle \in \mathbb{R}^{n-1}$, is also a time series.

Let $\omega \in \mathbb{R}^m, m \ll n$ be an ordered sequence of constant values called a **kernel**. A **feature map** $T * \omega \in \mathbb{R}^{n-m+1}$ is computed by applying a one-dimensional **convolution** operator $*$ to T and ω . A convolution $*_d$ with a **dilation** of d skips every $d - 1$ scalar elements of T during the convolution [22], [23]; when accounting for dilations $T *_d \omega \in \mathbb{R}^{(n-m+1)/d}$. Fig. 2 illustrates these concepts.

Let $\Omega = \langle \omega_1, \omega_2, \dots, \omega_N \rangle \in \mathbb{R}^{N \times m}$ be an ordered sequence of kernels. Each kernel is convolved with T using a discrete set of dilation values $D \subset \mathbb{N}^+$ to produce an ordered set of feature maps $T * \Omega = \langle T *_d \omega_i | 1 \leq i \leq N, d \in D \rangle$.

A **reduction** function $f : \mathbb{R}^{n-m+1} \rightarrow \mathbb{R}$ computes a scalar property of a feature map. A **pooling layer** computes the reduction for each feature map: $F(T * \Omega) = \langle f(T *_d \omega_i) | 1 \leq i \leq N, d \in D \rangle$, producing a **feature vector** for a downstream classifier. Recommended classifiers are **ridge regression** (for smaller datasets) and **logistic regression** (for larger datasets) [24].

B. Dilations

Computing a convolution using the same kernel but with different dilations captures the pattern at varying scales. Dilations commonly used include: (1) random dilations; (2) uniformly spaced dilations in $[1, \lfloor 2^p \rfloor]$, where $p = \log_2(n - 1)/8$; and (3) exponential dilations $\{2^0, 2^1, 2^2, \dots\}$.

C. Reduction Function

MiniRocket uses a single reduction function. Given a **bias** term $b > 0$, the **(multi-)set of positive values** of convolution $T * \omega$ with bias b is:

$$PV_b(T * \omega) = \{u_i \in T * \omega | u_i > b\} \quad (1)$$

The **Proportion of Positive Values (PPV)** is:

$$PPV_b(T * \omega) = \frac{1}{n - m + 1} |PV_b(T * \omega)| \quad (2)$$

D. Convolutional Kernel-based Feature Generators

1) **MiniRocket**: A kernel-based classifier parameterized with kernel length $m = 9$, kernels $\omega \in \{-1, 2\}^9$ with six values of -1 and three of 2 (i.e., $C(9, 3) = 84$ kernels), bias b taken from the convolution output, fixed padding and dilation, and uniformly-spaced dilations with a maximum of 32 dilations per kernel. MiniRocket uses the PPV reduction function exclusively to generate $\sim 10,000$ features: $F_{PPV}(T * \Omega)$.

2) **Hydra**: An alternative approach that computes histograms as features rather than pooling [7]. Hydra uses kernel length $m = 9$, random kernels $\omega \sim \{\mathcal{N}(0, 1)\}^{n-m+1}$, fixed padding, and exponential dilations. Hydra forms G groups of K kernels each. Let $\Omega^{(g)} = \langle \omega_1^{(g)}, \dots, \omega_K^{(g)} \rangle$ denote the g^{th} kernel group. For each group, Hydra computes convolutions $T *_d \Omega^{(g)}$ and $T' *_d \Omega^{(g)}$ for all $d \in D$.

Consider convolution $T *_d \omega_j^{(g)} = \langle x_{1,j}^{(g)}, \dots, x_{p,j}^{(g)} \rangle$, where p is the number of channels. The **maximum response** and **maximum response index** for channel i are:

$$\mathcal{R}_{max,i}(T *_d \Omega^{(g)}) = \max_{1 \leq k \leq K} (x_{i,k}^{(g)}) \quad (3)$$

$$\mathcal{I}_{max,i}(T *_d \Omega^{(g)}) = \arg \max_{1 \leq k \leq K} (x_{i,k}^{(g)}) \quad (4)$$

The **maximum channel set** for kernel $\omega_j^{(g)}$ is $\mathcal{C}_{max}(\omega_j^{(g)}) = \{i | \mathcal{I}_{max,i}(T *_d \Omega^{(g)}) = j\}$. Minimum responses and channel sets are defined analogously using $-\Omega^{(g)}$.

Hydra defines a **hard count** as the number of channels for which each kernel computes the maximum response, and a **soft count** as the sum of those maximum responses. The **minimum hard count histogram** and **maximum soft count histogram** are:

$$H_{min}(T *_d \Omega^{(g)}) = \left\langle \left| \mathcal{C}_{min}(\omega_j^{(g)}) \right| \mid 1 \leq j \leq p \right\rangle \quad (5)$$

$$S_{max}(T *_d \Omega^{(g)}) = \left\langle \sum_{i \in \mathcal{C}_{max}(\omega_j^{(g)})} x_{i,j}^{(g)} \mid 1 \leq j \leq p \right\rangle \quad (6)$$

Hydra generates features $H_{min}(T *_d \Omega^{(g)}) \cdot S_{max}(T *_d \Omega^{(g)})$ for $1 \leq g \leq G, d \in D$.

E. Regression Classifier

Let $F(T * \omega) \in \mathbb{R}^{|D|NM}$ be the feature vector produced by one of the kernel-based classifiers, **z-score normalized** to $F(T * \omega)$, and provided as input to ridge regression or logistic regression [24]. Both are linear classifiers with response function $F(T * \omega) \cdot W \in \mathbb{R}$, where $W \in \mathbb{R}^{|D|NM}$ is the set of trained weights.

Let $C = \langle C_1, C_2, \dots, C_P \rangle$ be the set of classes. The regression classifier comprises P independent linear classifiers with

weights $\langle W_1, W_2, \dots, W_P \rangle$, each predicting the probability that T belongs to each class. The predicted class is:

$$j = \arg \max_{1 \leq i \leq P} (\overline{F(T * \omega)} \cdot W_i). \quad (7)$$

III. FPGA ACCELERATOR ARCHITECTURES

We present two FPGA accelerator architectures, one for MiniRocket (Fig. 3) and the other for Hydra (Fig. 4)². Figs. 3 and 4 share a common interface. The time series being classified is streamed over the network; each data-point is a 32-bit float. The **CMAC** is a 100 Gbps Ethernet port and the **UDP** block implements the User Datagram Protocol; our testbed does not necessitate reliable transport. The accelerator can also read the time series from DDR or High-Bandwidth Memory (HBM) (not shown).

Time series datapoints arrive one-at-a-time over the network. The accelerator maintains two circular buffers to store the most recent window of sampled datapoints of time series T , along with T 's first-order difference series T' . When a new datapoint arrives, the oldest datapoints in T and T' are discarded and T' is updated by taking the difference between the two most recently sampled datapoints $t_2 - t_1$.

For both accelerators, the total amount of storage required for the two time series, kernels, feature maps, concatenated features, and regression classifier weights fit within the aggregate BRAM capacity of our target FPGA, an AMD Alveo U280. As such, BRAM storage is used exclusively, with most of the space allocated to the kernels.

Both accelerators include Convolution Engines, which are functionally identical. The MiniRocket Convolution Engines (Fig. 3) are sized to the number of kernels and dilations, whereas Hydra's Group Convolution Engines (Fig. 4) are sized to the number of kernels and dilations within a kernel group, and then replicated (two Convolution Engines per kernel group). MiniRocket's Pooling Engine computes the PPV feature exclusively, whereas Hydra's Group Pooling Engines compute the minimum hard count and maximum soft count histograms for each group. The regression classifier computes one dot product per class plus an $\arg \max$.

A. Convolution Engine Design

A convolution is a sliding dot product computed over time series $T \in \mathbb{R}^n$ with kernel $\omega \in \mathbb{R}^m$, $m < n$. Let $T_{i..j}$ denote the subsequence $\langle t_i, t_{i+1}, \dots, t_{i+j} \rangle$ of T ; to account for dilation, let $T_{i..j}^{(d)}$ denote subsequence $\langle t_i, t_{i+d}, \dots, t_{i+jd} \rangle$. In other words, $T * \omega = \langle T_{1..m}^{(d)} \cdot \omega, T_{2..m+1}^{(d)} \cdot \omega, \dots, T_{n-m+1..m}^{(d)} \cdot \omega \rangle$, is a sequence of dot products between ω and appropriately dilated subsequences of T , as shown in Fig. 2. The Convolution Engine maximizes efficiency through reuse of dot products.

In our context, T is a streaming time series in which new datapoints arrive at a fixed sampling rate, subject to network

variability. When new datapoint t_0 arrives, the oldest datapoint t_n is discarded, and indices are adjusted accordingly, $t_i \rightarrow t_{i+1}$, $0 \leq i \leq n-1$. Both T and the feature map $T * \Omega$ computed by the Convolution Engine are circular buffers.

For the purpose of analysis, we extend T to include t_0 and perform no index adjustment. The Convolution Engine first computes $T_{1..n} * \omega$ followed by $T_{0..n-1} * \omega$ once t_0 arrives. We observe that $T_{1..n-1} * \omega$, a sequence of $n-1$ dot products, is a common subsequence of both $T_{1..n} * \omega$ and $T_{0..n-1} * \omega$. To compute $T_{0..n-1} * \omega$, the Convolution Engine removes $T_{n-(m+1)d..n}^{(d)}$ from $T_{1..n} * \omega$ and prepends $T_{0..m-1}^{(d)} \cdot \omega$.

As shown in Fig. 5, the Convolution Engine must also account for padding (i.e., leading/trailing zeros.). Let $0^\alpha \cdot T \cdot 0^\beta$ denote T padded with $\alpha \geq 0$ leading zeros and $\beta \geq 0$ trailing zeros. When new datapoints arrive, only T is adjusted: when t_0 arrives and t_n is discarded, the padded time series transitions from $0^\alpha \cdot T_{1..n} \cdot 0^\beta$ to $0^\alpha \cdot T_{0..n-1} \cdot 0^\beta$. The Convolution Engine computes feature maps $0^\alpha \cdot T_{1..n} \cdot 0^\beta * \omega$ and $0^\alpha \cdot T_{0..n-1} \cdot 0^\beta * \omega$, noting that $T_{1..n-1} * \omega$ remains common to feature maps, but dot products involving the padding are not. Fig. 5 shows an example where $\alpha = 4$ and $\beta = 0$.

Each padded convolution decomposes into a padding-dependent prefix, a shared interior, and a padding-dependent suffix. After t_0 arrives, the Convolution Engine rewrites the new convolution to reuse $T_{1..n-1} * \omega$:

$$\begin{aligned} 0^\alpha \cdot T_{0..n-1} \cdot 0^\beta * \omega &= (0^\alpha \cdot T_{0..m-2} * \omega) \cdot \\ &\quad \langle T_{0..m-1} \cdot \omega \rangle \cdot (T_{1..n-1} * \omega) \cdot \\ &\quad (T_{m-n+1..n-1} \cdot 0^\beta * \omega). \end{aligned} \quad (8)$$

Without reuse, each convolution computes $\alpha + \beta + (n - m) + 1$ dot products, which is reduced to $\alpha + \beta + 1$ with reuse. Aggregated across all kernels and dilations, the number of dot products computed by the Convolution Engine is reduced from $|\Omega||D|(\alpha + \beta + (n - m) + 1)$ to $|\Omega||D|(\alpha + \beta + 1)$.

B. MiniRocket Floating-point Multiplication

All of the aforementioned classifiers are specified using 32-bit floating-point numbers and operators. A floating-point number is a tuple $\langle s, e, f \rangle$, where s is a **sign bit**, e is an **exponent** and f is the **fractional** component, or **mantissa**; a floating-point value is computed as $(-1)^s \cdot 2^{e-b} \cdot 1.f$, where b is a constant-valued bias term, and $1.f$ represents a fractional value in the range $[1, 2)$; special encodings are provided for ± 0 , $\pm \infty$, and not-a-number (*NaN*). A 32-bit **float** has 8 exponent bits and 23 mantissa bits.

In MiniRocket, each kernel ω consists of nine scalar values, six of which are -1 and three of which are 2 . Multiplication by -1 inverts the sign bit s and multiplication by 2 can be implemented by incrementing e (we set the bias b to 0), with appropriate control logic for when e overflows.

We propose a binary encoding for MiniRocket kernels: $\{0 \rightarrow -1, 1 \rightarrow 2\}$; this enables a compact floating-point multiplier design for encoded constants -1 and 2 . Consider floating-point operation $X \times Y$, where X is any value, and Y is either -1 or 2 ; we use lower-case y to represent the binary

²We did not design an accelerator for Rocket because of the inefficiencies which result from variable-length kernels: either an indexing table would be needed to identify the memory address of each kernel; or the kernels would need to be padded with zeroes so that all kernels have length 11.

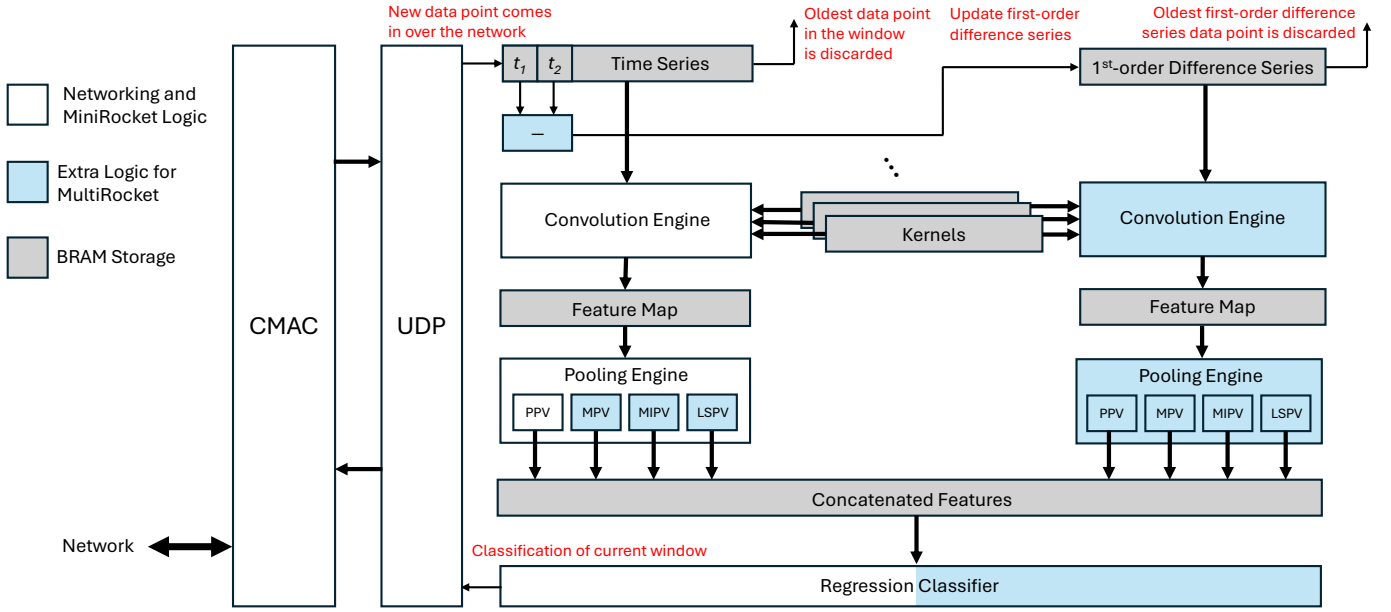


Fig. 3: FPGA accelerator for MiniRocket. Blue-shaded components support additional pooling operators for future work.

encoding of Y . Our proposed '**binary**' floating-point (**bFP**) multiplier is specified as:

$$\langle XNOR(X.s, y), X.e + y, X.f \rangle. \quad (9)$$

Convolution operators can compute dot products using the bFP multiplier, but must perform accumulation using standard floating-point accumulators.

In our experiments, we never observed an instance where multiplying by 2 caused the exponent to overflow, so we omit overflow handling logic as well as $\pm\infty$ and NaN .

C. Pooling and Classification Pipeline

After feature extraction produces an F -dimensional feature vector (where $F = 84 \times |D| \times P$ for $|D|$ dilations and P pooling operators), two additional pipeline stages complete the inference.

A **Scaler Kernel** normalizes each feature f_i to zero mean and unit variance: $\hat{f}_i = (f_i - \mu_i)/\sigma_i$, where μ_i and σ_i are precomputed from training data and stored in on-chip BRAM.

A **Classifier Kernel** computes ridge regression decision scores $s_c = \mathbf{w}_c^T \hat{\mathbf{f}} + b_c$ for each class c , selecting $\arg \max_c s_c$. The coefficient matrix $\mathbf{W} \in \mathbb{R}^{C \times F}$ and intercept vector $\mathbf{b} \in \mathbb{R}^C$ are precomputed on the host and transferred via HBM.

The Scaler and Classifier Kernels are connected to the Feature Extraction Kernel via AXI-Stream interfaces, forming a three-stage pipeline. Array partitioning distributes the feature map across multiple BRAM banks to enable concurrent write access by the 84 unrolled kernel loops. The pooling engine is pipelined rather than unrolled because the feature vector (~ 840 entries for MiniRocket) would require infeasible register resources if fully unrolled. For MiniRocket ($F = 840$), scaling and classification together consume <0.1 ms per sample, making feature extraction the dominant bottleneck (85–99% of total inference time).

IV. EXPERIMENTAL SETUP

A. FPGA Implementation Details

We implemented MiniRocket and Hydra inference accelerators shown in Figs. 3 and 4 targeting an AMD Alveo U280 FPGA with a maximum window size of 8,192³.

We specified the accelerators using HLS-compatible C++, and compiled them using Vitis HLS 2023.2 with synthesis and bitstream generation using Vivado 2023.2. We report resource utilization post place-and-route and achieved a 300 MHz target frequency in all cases.

We evaluated three MiniRocket Convolution Engine variants to isolate the impact of our proposed optimizations (Sections III-A and III-B):

- **Naïve + FP**: recomputes all dot products each time a new data point arrives using standard 32-bit floating-point multiplication;
- **Naïve + bFP**: introduces the bFP multiplier (Eq. 9);
- **DP-Reuse + bFP**: reuses dot products within the convolution (Fig. 5).

Additionally, we evaluate a **Fused CONV+PPV** kernel that combines convolution and pooling into a single pass, eliminating the intermediate feature-map array and achieving zero DSP utilization. The Hydra accelerator uses `ap_fixed<32,16>` arithmetic with UNROLL=16.

B. Software Baseline and Datasets

MiniRocket CPU baselines use a C++ implementation compiled with `-O3 -march=native` on the same Xeon E5-2640 v3; the Hydra CPU baseline uses single-threaded Python

³Smaller windows can be used as well. The large window provides a direct comparison to software, which classifies the entire time series in one-shot.

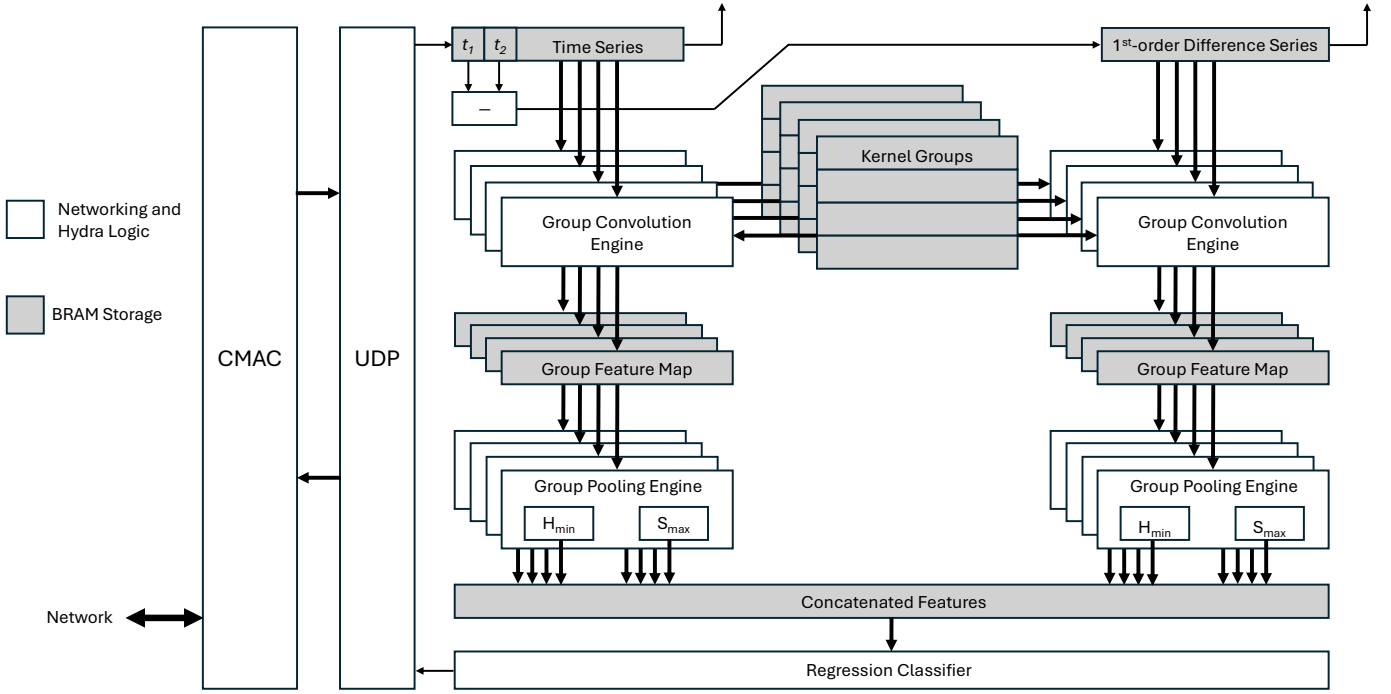


Fig. 4: FPGA accelerator for Hydra. Network streaming (CMAC/UDP) was evaluated for MiniRocket only (Section V-E); Hydra results use the single-node configuration.

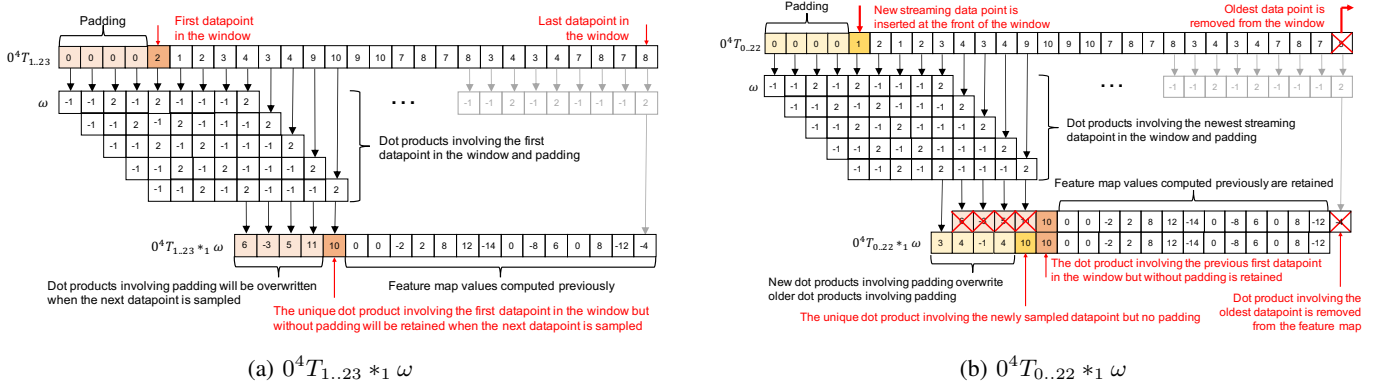


Fig. 5: Example of a convolution (dilation $d = 1$) with dot-product reuse applied to a front-padded streaming time series $0^4T_{0..23}$ before (a) and after (b) a new datapoint is added to the window.

3.10 from the Aeon-toolkit [25], [26], the reference implementation by the algorithm’s authors, as no optimized C++ Hydra implementation was available at time of writing. GPU baselines were measured on an NVIDIA Tesla T4 (16 GB, 70 W TDP), a widely deployed inference-class accelerator, using PyTorch 2.10.0 with CUDA 12.8; throughput is reported at batch sizes 256 and 1. All classifiers use ridge regression.

We selected the three longest time series from the UCR Time Series Archive [4]; Table I details their train/test splits and sample length. We trained all classifiers using Aeon and confirmed that our accelerators’ inference results match those of Aeon’s MiniRocket and Hydra (Table III).

C. Single-Node Experiments

CPU baselines ran on an Intel Xeon E5-2640 v3 @2.60GHz with 128 GiB DDR4. The FPGA accelerator ran on an Alveo U280 (2022.1 shell) with the same host. FPGA power was measured via `xbutil examine`; CPU power via Intel RAPL during sustained single-threaded inference. We fully unrolled the 84 kernel and dilation loops with array partition-

TABLE I: Time series datasets used in our experiments.

	Train	Test	Sample Length	# Classes
InsectSound	25,000	25,000	600	10
MosquitoSound	139,786	139,786	3,750	6
FruitFlies	17,259	17,259	5,000	3

TABLE II: Post place-and-route resource utilization on Alveo U280.

Algorithm	Config	FF	LUTs	DSPs	BRAMs	Freq
MiniRocket	Fused 1CU	94,283	57,090	0	185	300 MHz
MiniRocket	Fused 2CU	188,566	114,180	0	370	300 MHz
MiniRocket	Fused 3CU	159,086	92,703	1,264	428	269 MHz
MiniRocket	DP-Reuse 1CU	153,327	135,634	2	1,320	300 MHz
HYDRA	v2_fixed 1CU	58,537	41,746	1,355	86	300 MHz
HYDRA	Dataflow 1CU	20,823	15,866	55	115	405 MHz

TABLE III: Throughput (inferences/sec) and classification accuracy (%). Baseline = single-threaded C++ -O3 -march=native on the same kernel source compiled natively (HLS pragmas no-op); Speedup = FPGA 3CU / C++ baseline.

Algorithm	Dataset	Len	C++ Base	FPGA 1CU	FPGA 3CU	Speedup	Ref Acc	FPGA Acc
MiniRocket	InsectSound	600	289	1,967	5,012	17.3×	78.84	74.12
MiniRocket	MosquitoSound	3,750	48	937	2,820	58.7×	87.59	87.88
MiniRocket	FruitFlies	5,000	258	742	2,191	8.5×	96.98	95.82
HYDRA	InsectSound	600	1,372	6,326	10,573	7.7×	69.41	69.41
HYDRA	MosquitoSound	3,750	326	1,937	3,516	10.8×	70.05	70.05
HYDRA	FruitFlies	5,000	245	1,507	2,972	12.1×	87.61	87.61

Ref Acc = full-precision (float64) reference accuracy on the full test set. MiniRocket FPGA accuracy differences ($\leq 5\%$) arise from float64→float32 model export; HYDRA matches exactly.

ing for concurrent BRAM access. The Pooling Engine and Regression Classifier are pipelined.

D. Streaming Experiment

The FPGA is configured as a SmartNIC integrating the CMAC and UDP modules in Fig. 3. We additionally measured CPU RTT using DPDK kernel bypass. The experiment ran on the Open-Cloud Testbed (OCT) [27], [28] accessed via CloudLabs [29], a cluster of 24 Alveo U280s connected by 100GbE switches. A CPU client transmits UDP packets to the FPGA inference server; timing is captured as RTT from transmission to prediction receipt, using the ArrowHead time series (251 samples, 3 classes).

V. EXPERIMENTAL RESULTS

A. Resource Utilization

Table II reports post place-and-route resource utilization on the Alveo U280. All single-node designs achieve 300 MHz; the 3-CU MiniRocket variant is auto-throttled to 269 MHz due to SLR crossing penalties.

MiniRocket’s fused CONV+PPV kernel achieves zero DSP utilization by exploiting the ternary weight structure ($\{-1, +2\}$) through the bFP multiplier (Eq. 9). Hydra’s v2_fixed variant uses 1,355 DSPs because its learned random kernels require standard floating-point multiplication, implemented via ap_fixed(32,16) arithmetic.

B. Throughput, Latency, and Accuracy

Table III reports throughput and classification accuracy. Both MiniRocket and Hydra are compared against an optimized single-threaded C++ baseline (-O3 -march=native). For Hydra, no third-party optimized C++ implementation exists; we therefore compile the Hydra

TABLE IV: Impact of fused CONV+PPV kernel on MiniRocket (1 CU, 300 MHz).

	Separate	Fused	Improvement
<i>Throughput (inferences/sec)</i>			
InsectSound	295.9	1,966.8	6.6×
MosquitoSound	88.1	937.0	10.6×
FruitFlies	67.9	742.3	10.9×
<i>Resource Utilization (HLS synthesis)</i>			
LUTs	48,812	57,090	+17%
FFs	88,492	94,283	+7%
BRAM (18K)	576	185	-68%
DSP48E2	724	0	-100%
Intermediate array	16 × 8192 floats	Eliminated	-100%

TABLE V: MiniRocket CU scaling (fused kernel, inferences/sec).

Dataset	1 CU	2 CU	3 CU	2/1 Scaling	3/1 Scaling
InsectSound	1,966.8	3,191.1	5,012.2	1.62×	2.55×
MosquitoSound	937.0	1,496.9	2,819.6	1.60×	3.01×
FruitFlies	742.3	1,121.4	2,190.7	1.51×	2.95×

HLS kernel sources directly as native C++ (HLS pragmas are no-ops outside Vitis HLS), which yields a fair single-core CPU reference. Reference accuracy on a 1000-sample subset matches the FPGA full-set accuracy within statistical noise.

The fused MiniRocket kernel achieves 742–1,967 inferences/sec (1 CU), corresponding to 8.5–58.7× speedup over C++ with 3 CUs. Hydra achieves 1,507–6,326 inferences/sec (1 CU), scaling to 2,972–10,573 with 3 CUs (7.7–12.1× over the same C++ baseline). Hydra’s narrower speedup ratio reflects that its monolithic kernel maps more efficiently onto a CPU than MiniRocket’s heavily unrolled 84-kernel pipeline. Single-CU P50 latency ranges from 0.62 ms to 1.49 ms for MiniRocket and 0.16 ms to 0.66 ms for Hydra.

C. Design Space Exploration

Table IV quantifies the impact of fusing convolution and PPV pooling into a single-pass kernel. Fusion eliminates the intermediate convolutions[16][8192] array, reducing BRAM by 68% and DSP to zero, while achieving 6.6–10.9× throughput improvement.

Table V reports throughput scaling from 1 to 3 compute units. The 2-CU design meets timing at 300 MHz with 1.51–1.62× scaling; the 3-CU design uses a higher unroll factor (28 vs. 16) to compensate for SLR crossing latency, which introduces 1,264 DSPs per CU (Table II), and achieves 2.55–3.01× scaling despite being auto-throttled to 269 MHz due to timing closure failure (WNS=−0.38 ns).

Table VI compares the monolithic Hydra kernel against a 3-kernel on-chip dataflow decomposition connected by AXI-Stream interfaces (load → hydra_inference → store). This on-chip partitioning (distinct from the network streaming experiment in Section V-E) synthesizes at 404.86 MHz and matches the Python reference exactly on all three datasets, confirming that the architecture is amenable to modular partitioning. However, end-to-end throughput is

TABLE VI: HYDRA FPGA design comparison: monolithic kernel vs. 3-kernel dataflow pipeline (on-chip AXI-Stream). Both use `ap_fixed<32,16>`, UNROLL=16, 1CU on Alveo U280.

Dataset	Monolithic (v2_fixed)			Dataflow (3-kernel)		
	Acc (%)	Lat (ms)	Tput (inf/s)	Acc (%)	Lat (ms)	Tput (inf/s)
InsectSound	69.41	0.158	6,326	69.41	5.19	193*
MosquitoSound	70.05	0.516	1,937	70.05	32.1	31*
FruitFlies	87.61	0.663	1,507	87.61	42.7	23*

*Streaming throughput limited by per-sample OpenCL dispatch, not kernel compute. Both variants match the Python reference exactly ($\Delta=0.00\%$).

TABLE VII: GPU (Tesla T4) vs. FPGA (Alveo U280) throughput (inferences/sec). FPGA columns use per-sample OpenCL dispatch (effective batch=1); GPU columns report both batch=256 (offline upper bound) and batch=1 (streaming).

Algorithm	Dataset	GPU b=256 (inf/s)	GPU b=1 (inf/s)	FPGA 1CU (inf/s)	FPGA 3CU (inf/s)	GPU/FPGA (b=256/3CU)
MiniRocket	InsectSound (600)	30,469	1,091	1,967	5,012	6.1×
MiniRocket	MosquitoSound (3750)	4,198	661	937	2,820	1.5×
MiniRocket	FruitFlies (5000)	3,227	539	742	2,191	1.5×
HYDRA	InsectSound (600)	13,895	2,711	6,326	10,573	1.3×
HYDRA	MosquitoSound (3750)	5,139	2,805	1,937	3,516	1.5×
HYDRA	FruitFlies (5000)	4,267	1,934	1,507	2,972	1.4×

limited by per-sample OpenCL dispatch; for batch inference the monolithic variant remains preferable.

D. GPU Comparison

Table VII reports throughput at two batch sizes that bracket the GPU’s operating envelope. Batch size 256 is the GPU’s best case for offline/bulk inference, where many samples are available simultaneously. Batch size 1 is the per-sample regime targeted by this work: streaming sensor classification (Section V-E) and any latency-bound deployment in which samples arrive sequentially over a network or sensor interface, since batching b samples adds at minimum $(b-1) \cdot \Delta t$ of queueing delay relative to immediate dispatch. We report both endpoints rather than a single operating point. At batch=256, the GPU achieves 1.5–6.1 $\times$ higher throughput than the 3-CU FPGA for MiniRocket (advantage largest for short series, where GPU parallelism dominates) and 1.3–1.5 $\times$ for Hydra. At batch=1, the FPGA 3-CU configuration exceeds GPU throughput on all datasets for both algorithms.

Fig. 6 shows how the GPU/FPGA throughput ratio varies with time series length. The GPU advantage at batch=256 drops from 65 $\times$ at length 64 to 3.8 $\times$ at length 8,192. At batch=1, the FPGA matches the GPU at length $\sim 4,096$ (single CU) and $\sim 2,048$ (3-CU).

Table VIII compares energy efficiency across platforms. The FPGA consumes 25.7 W (measured via `xbutil examine`); the GPU consumes 54–68 W under sustained inference; CPU power is measured via Intel RAPL at 40–53 W under single-threaded inference. For long time series (MosquitoSound), the FPGA 3-CU configuration achieves 92 $\times$ better energy efficiency than the CPU baseline and 1.4 $\times$ better than the GPU at batch=256. For short series (InsectSound), the GPU’s higher batched throughput yields 2.3 $\times$ better energy efficiency; the FPGA advantage re-emerges at batch=1. Hydra exhibits simi-

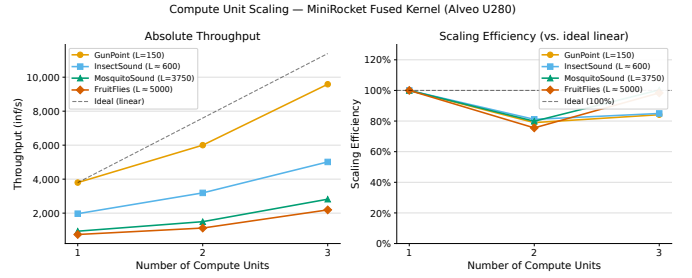


Fig. 6: MiniRocket GPU/FPGA throughput ratio vs. time series length. GPU: Tesla T4, batch=256 and batch=1. FPGA: Alveo U280, fused 1-CU, 300 MHz.

TABLE VIII: Energy efficiency: MiniRocket inference. FPGA 25.7 W (static-dominated at <5% utilization); GPU via `nvidia-smi`; CPU via RAPL.

Dataset	Platform	Tput (inf/s)	Power (W)	mJ/inf	vs CPU
InsectSound	CPU C++	289	52.8	182.7	1×
	GPU T4 (b=256)	30,469	68	2.2	83×
	FPGA 3-CU	5,012	25.7	5.1	36×
MosquitoSound	CPU C++	48	40.0	833.3	1×
	GPU T4 (b=256)	4,198	54	12.9	65×
	FPGA 3-CU	2,820	25.7	9.1	92×
FruitFlies	CPU C++	258	49.9	193.4	1×
	GPU T4 (b=256)	3,227	54	16.7	12×
	FPGA 3-CU	2,191	25.7	11.7	17×

lar trends: at 25.7 W, the 3-CU FPGA consumes 2.4–8.6 mJ/inf across datasets.

E. Network Streaming Results

In the MiniRocket SmartNIC experiment (ArrowHead dataset), the FPGA achieves 4.12 ms average RTT (1.25 ms computation + 2.87 ms communication) vs. 11.0 ms for CPU with Python sockets (2.65 $\times$). With DPDK kernel bypass, CPU RTT drops to 5.66 ms (2.31 ms computation + 3.35 ms communication); the FPGA retains a 1.37 $\times$ advantage, with gains in both computation (1.85 $\times$) and communication (1.17 $\times$).

VI. RELATED WORK

A. Time Series Classification

Research in time series classification is supported by public archives [4]–[6] and software toolkits [25], [26], [30]–[32]. Two bake-offs [1]–[3] compared state-of-the-art classifiers; both identified convolution-based methods among the best.

Rocket [24] introduced untrained convolutional kernels with lengths $m \in \{7, 9, 11\}$ and random weights. MiniRocket [9] simplified Rocket using fixed-length ternary kernels ($m = 9$, $C(9, 3) = 84$ kernels). MultiRocket [33] extended MiniRocket with first-order difference features and additional reduction functions. **Hydra** [7] uses random kernels organized into groups with histogram-based pooling. The 2024 bake-off identified **MultiRocket+Hydra** and **HIVE-COTE 2.0** [34] as best-in-class. This paper presents FPGA accelerators for MiniRocket and Hydra; accelerators for the remaining HIVE-COTE 2.0 classifiers are left for future work.

B. FPGA-accelerated Time Series Computation

To the best of our knowledge, this is the first work to investigate hardware acceleration for any time series classifier evaluated in the 2017 or 2024 bake-offs. Prior FPGA work has targeted subroutines used by other classifiers: data correlation [35], shapelets [36], [37], Dynamic Time Warping [38], [39], the Matrix Profile [40], [41], and power spectra [42]. Eciton [43] accelerates an LSTM-based classifier on FPGA but was not evaluated on the bake-off datasets. The 2024 bake-off considered several CNNs [20], [44], [45], identifying **H-InceptionTime** [44] as best among deep-learning approaches.

Unlike CNN-based FPGA accelerators that map learned floating-point convolutions to DSP arrays, our work targets classifiers with fixed (MiniRocket) or untrained (Hydra) kernels. MiniRocket’s ternary weights enable zero-DSP multiplication, and the entire pipeline maps to a single-pass dataflow architecture without layer-by-layer scheduling.

VII. CONCLUSION

This paper has shown that FPGAs can accelerate time series classification using MiniRocket and Hydra in both single-node and network streaming contexts, with accelerator accuracy matching the reference CPU implementation within 0.01% on all benchmark datasets. MiniRocket’s fused CONV+PPV kernel achieves $8.5\text{--}58.7\times$ speedup over an optimized C++ baseline using up to three compute units. Compared to a Tesla T4 GPU, the 3-CU FPGA exceeds batch-size-one throughput on all evaluated datasets while consuming $2.1\text{--}2.6\times$ less power, with sub-0.5 ms mean latency across all datasets. An on-chip dataflow decomposition of the Hydra kernel via AXI-Stream interfaces preserves bit-exact accuracy, confirming that the architecture is amenable to modular partitioning. In a network streaming experiment, the FPGA SmartNIC achieves $1.37\times$ lower RTT than a DPDK-optimized CPU baseline. Multi-FPGA scaling via 100G Ethernet is under active investigation.

REFERENCES

- [1] A. J. Bagnall, J. Lines, A. Bostrom, J. Large, and E. J. Keogh, “The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances,” *Data Min. Knowl. Discov.*, vol. 31, no. 3, pp. 606–660, 2017. [Online]. Available: <https://doi.org/10.1007/s10618-016-0483-9>
- [2] M. Middlehurst, P. Schäfer, and A. J. Bagnall, “Bake off redux: a review and experimental evaluation of recent time series classification algorithms,” *Data Min. Knowl. Discov.*, vol. 38, no. 4, pp. 1958–2031, 2024. [Online]. Available: <https://doi.org/10.1007/s10618-024-01022-1>
- [3] —, “Correction: Bake off redux: a review and experimental evaluation of recent time series classification algorithms,” *Data Min. Knowl. Discov.*, vol. 38, no. 6, pp. 4236–4237, 2024. [Online]. Available: <https://doi.org/10.1007/s10618-024-01040-z>
- [4] H. A. Dau, A. J. Bagnall, K. Kamgar, C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. J. Keogh, “The UCR time series archive,” *IEEE CAA J. Autom. Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019. [Online]. Available: <https://doi.org/10.1109/jas.2019.1911747>
- [5] A. J. Bagnall, H. A. Dau, J. Lines, M. Flynn, J. Large, A. Bostrom, P. Southam, and E. J. Keogh, “The UEA multivariate time series classification archive, 2018,” *CoRR*, vol. abs/1811.00075, 2018. [Online]. Available: <http://arxiv.org/abs/1811.00075>
- [6] A. Dempster, N. M. Foumani, C. W. Tan, L. Miller, A. Mishra, M. Salehi, C. Pelletier, D. F. Schmidt, and G. I. Webb, “MONSTER: monash scalable time series evaluation repository,” *CoRR*, vol. abs/2502.15122, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2502.15122>
- [7] A. Dempster, D. F. Schmidt, and G. I. Webb, “Hydra: competing convolutional kernels for fast and accurate time series classification,” *Data Min. Knowl. Discov.*, vol. 37, no. 5, pp. 1779–1805, 2023. [Online]. Available: <https://doi.org/10.1007/s10618-023-00939-3>
- [8] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009)*, 20–25 June 2009, Miami, Florida, USA. IEEE Computer Society, 2009, pp. 248–255. [Online]. Available: <https://doi.org/10.1109/CVPR.2009.5206848>
- [9] A. Dempster, D. F. Schmidt, and G. I. Webb, “Minirocket: A very fast (almost) deterministic transform for time series classification,” in *KDD ’21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14–18, 2021*, F. Zhu, B. C. Ooi, and C. Miao, Eds. ACM, 2021, pp. 248–257. [Online]. Available: <https://doi.org/10.1145/3447548.3467231>
- [10] K. Jarrett, K. Kavukcuoglu, M. Ranzato, and Y. LeCun, “What is the best multi-stage architecture for object recognition?” in *IEEE 12th International Conference on Computer Vision, ICCV 2009, Kyoto, Japan, September 27 - October 4, 2009*. IEEE Computer Society, 2009, pp. 2146–2153. [Online]. Available: <https://doi.org/10.1109/ICCV.2009.5459469>
- [11] N. Pinto, D. Doukhan, J. J. DiCarlo, and D. D. Cox, “A high-throughput screening approach to discovering good forms of biologically inspired visual representation,” *PLoS Comput. Biol.*, vol. 5, no. 11, 2009. [Online]. Available: <https://doi.org/10.1371/journal.pcbi.1000579>
- [12] D. D. Cox and N. Pinto, “Beyond simple features: A large-scale feature search approach to unconstrained face recognition,” in *Ninth IEEE International Conference on Automatic Face and Gesture Recognition (FG 2011)*, Santa Barbara, CA, USA, 21–25 March 2011. IEEE Computer Society, 2011, pp. 8–15. [Online]. Available: <https://doi.org/10.1109/FG.2011.5771385>
- [13] A. M. Saxe, P. W. Koh, Z. Chen, M. Bhand, B. Suresh, and A. Y. Ng, “On random weights and unsupervised feature learning,” in *Proceedings of the 28th International Conference on Machine Learning, ICML 2011, Bellevue, Washington, USA, June 28 - July 2, 2011*, L. Getoor and T. Scheffer, Eds. Omnipress, 2011, pp. 1089–1096. [Online]. Available: https://icml.cc/2011/papers/551_icmlpaper.pdf
- [14] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” *CoRR*, vol. abs/1411.1792, 2014. [Online]. Available: <http://arxiv.org/abs/1411.1792>
- [15] Z. Cui, W. Chen, and Y. Chen, “Multi-scale convolutional neural networks for time series classification,” *CoRR*, vol. abs/1603.06995, 2016. [Online]. Available: <http://arxiv.org/abs/1603.06995>
- [16] Z. Wang, W. Yan, and T. Oates, “Time series classification from scratch with deep neural networks: A strong baseline,” in *2017 International Joint Conference on Neural Networks, IJCNN 2017, Anchorage, AK, USA, May 14–19, 2017*. IEEE, 2017, pp. 1578–1585. [Online]. Available: <https://doi.org/10.1109/IJCNN.2017.7966039>
- [17] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P. Muller, “Deep neural network ensembles for time series classification,” in *International Joint Conference on Neural Networks, IJCNN 2019 Budapest, Hungary, July 14–19, 2019*. IEEE, 2019, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/IJCNN.2019.8852316>
- [18] —, “Deep learning for time series classification: a review,” *Data Min. Knowl. Discov.*, vol. 33, no. 4, pp. 917–963, 2019. [Online]. Available: <https://doi.org/10.1007/s10618-019-00619-1>
- [19] Z. Zimmerman, N. S. Senobari, G. J. Funning, E. E. Papalexakis, S. Oymak, P. Brisk, and E. J. Keogh, “Matrix profile XVIII: time series mining in the face of fast moving streams using a learned approximate matrix profile,” in *2019 IEEE International Conference on Data Mining, ICDM 2019, Beijing, China, November 8–11, 2019*, J. Wang, K. Shim, and X. Wu, Eds. IEEE, 2019, pp. 936–945. [Online]. Available: <https://doi.org/10.1109/ICDM.2019.00104>
- [20] H. I. Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P. Muller, and F. Petitjean, “Inceptiontime: Finding alexnet for time series classification,” *Data Min. Knowl. Discov.*, vol. 34, no. 6, pp. 1936–1962, 2020. [Online]. Available: <https://doi.org/10.1007/s10618-020-00710-y>

- [21] H. Rakhshani, H. I. Fawaz, L. Idoumghar, G. Forestier, J. Lepagnet, J. Weber, M. Bréviliers, and P. Muller, "Neural architecture search for time series classification," in *2020 International Joint Conference on Neural Networks, IJCNN 2020, Glasgow, United Kingdom, July 19-24, 2020*. IEEE, 2020, pp. 1-8. [Online]. Available: <https://doi.org/10.1109/IJCNN48605.2020.9206721>
- [22] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016. [Online]. Available: <http://arxiv.org/abs/1511.07122>
- [23] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *CoRR*, vol. abs/1803.01271, 2018. [Online]. Available: <http://arxiv.org/abs/1803.01271>
- [24] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels," *Data Min. Knowl. Discov.*, vol. 34, no. 5, pp. 1454-1495, 2020. [Online]. Available: <https://doi.org/10.1007/s10618-020-00701-z>
- [25] M. Middlehurst, A. Ismail-Fawaz, A. Guillaume, C. Holder, D. Guijo-Rubio, G. Bulatova, L. Tsaprounis, L. Mentel, M. Walter, P. Schäfer, and A. J. Bagnall, "aeon: a python toolkit for learning from time series," *CoRR*, vol. abs/2406.14231, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2406.14231>
- [26] M. Middlehurst, A. J. Bagnall, G. Forestier, A. Ismail-Fawaz, and A. Guillaume, "Time series machine learning with aeon: Classification and regression," in *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track and Demo Track - European Conference, ECML PKDD 2025, Porto, Portugal, September 15-19, 2025, Proceedings, Part X*, ser. Lecture Notes in Computer Science, I. Dutra, M. Pechenizkiy, P. Cortez, S. Pashami, A. Pasquali, N. Moniz, A. M. Jorge, C. Soares, P. H. Abreu, and J. Gama, Eds., vol. 16022. Springer, 2025, pp. 432-437. [Online]. Available: https://doi.org/10.1007/978-3-032-06129-4_26
- [27] M. Zink, D. Irwin, E. Cecchet, H. Saplakoglu, O. Krieger, M. Herboldt, M. Daitzman, P. Desnoyers, M. Leeser, and S. Handagala, "The open cloud testbed (oct): A platform for research into new cloud technologies," in *2021 IEEE 10th International Conference on Cloud Networking (CloudNet)*, 2021, pp. 140-147.
- [28] S. Handagala, M. Leeser, K. Patle, and M. Zink, "Network attached fpgas in the open cloud testbed (oct)," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2022, pp. 1-6.
- [29] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra, "The design and operation of CloudLab," in *Proceedings of the USENIX Annual Technical Conference (ATC)*, Jul. 2019, pp. 1-14. [Online]. Available: <https://www.flux.utah.edu/paper/duplyakin-atc19>
- [30] M. Löning, A. J. Bagnall, S. Ganesh, V. Kazakov, J. Lines, and F. J. Király, "sktime: A unified interface for machine learning with time series," *CoRR*, vol. abs/1909.07872, 2019. [Online]. Available: <http://arxiv.org/abs/1909.07872>
- [31] J. Faouzi and H. Janati, "pyts: A python package for time series classification," *J. Mach. Learn. Res.*, vol. 21, pp. 46:1-46:6, 2020. [Online]. Available: <https://www.jmlr.org/papers/v21/19-763.html>
- [32] R. Tavenard, J. Faouzi, G. Vandewiele, F. Divo, G. Androz, C. Holtz, M. Payne, R. Yurchak, M. Rußwurm, K. Kolar, and E. Woods, "Tslern, A machine learning toolkit for time series data," *J. Mach. Learn. Res.*, vol. 21, pp. 118:1-118:6, 2020. [Online]. Available: <https://jmlr.org/papers/v21/20-091.html>
- [33] C. W. Tan, A. Dempster, C. Bergmeir, and G. I. Webb, "Multirocket: multiple pooling operators and transformations for fast and effective time series classification," *Data Min. Knowl. Discov.*, vol. 36, no. 5, pp. 1623-1646, 2022. [Online]. Available: <https://doi.org/10.1007/s10618-022-00844-1>
- [34] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. J. Bagnall, "HIVE-COTE 2.0: a new meta ensemble for time series classification," *Mach. Learn.*, vol. 110, no. 11, pp. 3211-3243, 2021. [Online]. Available: <https://doi.org/10.1007/s10994-021-06057-9>
- [35] M. Chiosa, T. B. Preußner, M. Blott, and G. Alonso, "AMNES: accelerating the computation of data correlation using fpgas," *Proc. VLDB Endow.*, vol. 16, no. 13, pp. 4174-4187, 2023. [Online]. Available: <https://www.vldb.org/pvldb/vol16/p4174-chiosa.pdf>
- [36] V. O. Costa, C. G. de Araujo Gewehr, J. C. Vicenzi, E. A. Carara, and L. L. de Oliveira, "Dedicated shapelet distance engine for time-series classification," *IEEE Des. Test*, vol. 39, no. 2, pp. 7-14, 2022. [Online]. Available: <https://doi.org/10.1109/MDAT.2021.3102884>
- [37] Q. Liu, W. Chen, and V. Dinavahi, "Multidimensional time-series shapelet-based real-time fault detection and localization on ISS electrical power distribution system," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1-12, 2024. [Online]. Available: <https://doi.org/10.1109/TIM.2023.3336447>
- [38] D. Sart, A. Mueen, W. A. Najjar, E. J. Keogh, and V. Niennattrakul, "Accelerating dynamic time warping subsequence search with gpus and fpgas," in *ICDM 2010, The 10th IEEE International Conference on Data Mining, Sydney, Australia, 14-17 December 2010*, G. I. Webb, B. Liu, C. Zhang, D. Gunopulos, and X. Wu, Eds. IEEE Computer Society, 2010, pp. 1001-1006. [Online]. Available: <https://doi.org/10.1109/ICDM.2010.21>
- [39] Z. Wang, S. Huang, L. Wang, H. Li, Y. Wang, and H. Yang, "Accelerating subsequence similarity search based on dynamic time warping distance with FPGA," in *The 2013 ACM/SIGDA International Symposium on Field Programmable Gate Arrays, FPGA '13, Monterey, CA, USA, February 11-13, 2013*, B. L. Hutchings and V. Betz, Eds. ACM, 2013, pp. 53-62. [Online]. Available: <https://doi.org/10.1145/2435264.2435277>
- [40] J. C. Romero, A. G. Navarro, A. Vilches, A. Rodríguez, F. Corbera, and R. Asenjo, "Efficient heterogeneous matrix profile on a CPU + high performance FPGA with integrated HBM," *Future Gener. Comput. Syst.*, vol. 125, pp. 10-23, 2021. [Online]. Available: <https://doi.org/10.1016/j.future.2021.06.025>
- [41] A. Kalantar, Z. Zimmerman, and P. Brisk, "Fpga-based acceleration of time series similarity prediction: From cloud to edge," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 1, pp. 12:1-12:27, 2023. [Online]. Available: <https://doi.org/10.1145/3555810>
- [42] D. J. M. Moss, Z. Zhang, N. J. Fraser, and P. H. W. Leong, "An fpga-based spectral anomaly detection system," in *2014 International Conference on Field-Programmable Technology, FPT 2014, Shanghai, China, December 10-12, 2014*, J. Chen, W. Yin, Y. Shibata, L. Wang, H. K. So, and Y. Ma, Eds. IEEE, 2014, pp. 175-182. [Online]. Available: <https://doi.org/10.1109/FPT.2014.7082772>
- [43] J. Chen, S. Jun, S. Hong, W. He, and J. Moon, "Eciton: Very low-power recurrent neural network accelerator for real-time inference at the edge," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, no. 1, pp. 16:1-16:25, 2024. [Online]. Available: <https://doi.org/10.1145/3629979>
- [44] A. Ismail-Fawaz, M. Devanne, J. Weber, and G. Forestier, "Deep learning for time series classification using new hand-crafted convolution filters," in *IEEE International Conference on Big Data, Big Data 2022, Osaka, Japan, December 17-20, 2022*, S. Tsumoto, Y. Ohsawa, L. Chen, D. V. den Poel, X. Hu, Y. Motomura, T. Takagi, L. Wu, Y. Xie, A. Abe, and V. Raghavan, Eds. IEEE, 2022, pp. 972-981. [Online]. Available: <https://doi.org/10.1109/BigData55660.2022.10020496>
- [45] A. Ismail-Fawaz, M. Devanne, S. Berretti, J. Weber, and G. Forestier, "LITE: light inception with boosting techniques for time series classification," in *10th IEEE International Conference on Data Science and Advanced Analytics, DSAA 2023, Thessaloniki, Greece, October 9-13, 2023*. IEEE, 2023, pp. 1-10. [Online]. Available: <https://doi.org/10.1109/DSAA60987.2023.10302569>